Interface ve Abstract: 10 Senaryo

Nov 6, 2026 · @Rabia

Nasıl çalışılır

Her senaryoda bir abstract sınıf, en az bir interface ve bunlardan türeyen sınıflar var. UML diyagramına bakıp sınıfları Java ile yazın, sonra `Main` içinde nesneleri oluşturun. Programınızın çıktısı **Beklenen çıktı** ile birebir aynı olmalı.

1. Her senaryoyu tek bir `.java` dosyasına yazın. Yalnızca `Main` sınıfı `public` olsun, diğer sınıfların başına `public` yazmayın.
2. Alan ve metot adlarını diyagramdaki gibi yazın; Türkçe karakter kullanmayın.
3. Alt sınıfın yapıcısında üst sınıfın yapıcısını `super(...)` ile çağırın.
4. Abstract metotları ve interface metotlarını ezdiğiniz her yerde `@Override` yazın.
5. Her senaryonun sonundaki **Düşün** sorusunu defterinize cevaplayın.

UML okuma anahtarı

Gösterim	Anlamı	Java'da
`<<abstract>>`	Abstract sınıf, nesnesi oluşturulamaz	`abstract class Hayvan`
`<<interface>>`	Interface, yalnızca metot başlıkları	`interface Yuzebilir`
Metodun sonunda `*`	Abstract metot, gövdesi yok	`public abstract String sesCikar();`
`+`	public	`public`
`#`	protected	`protected`
`-`	private	`private`
Düz çizgi, içi boş üçgen ok	Kalıtım	`extends`
Kesik çizgi, içi boş üçgen ok	Interface'i gerçekleştirme	`implements`
`sure(int km) int`	Parametre `int km`, dönüş tipi `int`	`public int sure(int km)`

Senaryo 1: Hayvanat Bahçesi

Hayvanat bahçesi, ziyaretçilere hayvanları tanıtan bir program istiyor. Her hayvanın bir adı, yaşı ve kendine özgü bir sesi var. Bazı hayvanlar yüzebiliyor, bazıları uçabiliyor, ördeğin ikisi de yapabiliyor.

```
classDiagram
    class Hayvan {
        <<abstract>>
        #String ad
        #int yas
        +Hayvan(String ad, int yas)
        +sesCikar()* String
        +bilgi() String
    }
    class Yuzebilir {
        <<interface>>
        +yuz() String
    }
    class Ucabilir {
        <<interface>>
        +uc() String
    }
    class Aslan {
        +Aslan(String ad, int yas)
        +sesCikar() String
    }
    class Penguen {
        +Penguen(String ad, int yas)
        +sesCikar() String
        +yuz() String
    }
    class Ordek {
        +Ordek(String ad, int yas)
        +sesCikar() String
        +yuz() String
        +uc() String
    }
    Hayvan <|-- Aslan
    Hayvan <|-- Penguen
    Hayvan <|-- Ordek
    Yuzebilir <|.. Penguen
    Yuzebilir <|.. Ordek
    Ucabilir <|.. Ordek
```

Görevler

1. `Yuzebilir` ve `Ucabilir` interface'lerini yazın.
2. Abstract `Hayvan` sınıfını yazın. `bilgi()` şunu döndürsün: `ad + " (" + yas + " yaşında)"`
3. `Aslan` sınıfı: `sesCikar()` → `"Kükreme!"`
4. `Penguen` sınıfı: `sesCikar()` → `"Gak gak!"`, `yuz()` → `ad + " buzlu suda yüzüyor."`
5. `Ordek` sınıfı: `sesCikar()` → `"Vak vak!"`, `yuz()` → `ad + " gölde yüzüyor."`, `uc()` → `ad + " gölün üstünde uçuyor."`
6. `Main` içine `static void tanit(Hayvan h)` metodunu yazın. Ekrana `h.bilgi() + " -> " + h.sesCikar()` yazsın.

Main'de yapılacaklar

1. Nesneler: `Aslan("Kral", 5)`, `Penguen("Buzi", 3)`, `Ordek("Paytak", 2)`
2. Üç nesneyi de `tanit` metoduna gönderin.
3. Penguen ve ördek nesnelerini `Yuzebilir` tipinde iki değişkene atayıp `yuz()` çağırın.
4. Ördeği `Ucabilir` tipinde bir değişkene atayıp `uc()` çağırın.

Beklenen çıktı

```
Kral (5 yaşında) -> Kükreme!
Buzi (3 yaşında) -> Gak gak!
Paytak (2 yaşında) -> Vak vak!
Buzi buzlu suda yüzüyor.
Paytak gölde yüzüyor.
Paytak gölün üstünde uçuyor.
```

Düşün: `Main` içine `Hayvan h = new Hayvan("X", 1);` yazın. Hangi hatayı alıyorsunuz, neden?

Senaryo 2: Şekil Atölyesi

Bir çizim programı farklı şekillerin alanını ve çevresini hesaplıyor. Her şeklin hesabı farklı olduğu için bu metotlar abstract. Kare aslında kenarları eşit bir dikdörtgen, bu yüzden `Kare` sınıfı `Dikdortgen`'den türüyor. Dikdörtgenler boyanabiliyor.

```
classDiagram
    class Sekil {
        <<abstract>>
        #String ad
        +Sekil(String ad)
        +alan()* int
        +cevre()* int
```

```
        +bilgi() String
    }
    class Boyanabilir {
        <<interface>>
        +boya(String renk) void
        +getRenk() String
    }
    class Dikdortgen {
        #int en
        #int boy
        -String renk
        +Dikdortgen(int en, int boy)
        +alan() int
        +cevre() int
        +boya(String renk) void
        +getRenk() String
    }
    class Kare {
        +Kare(int kenar)
    }
    class DikUcgen {
        -int a
        -int b
        -int c
        +DikUcgen(int a, int b, int c)
        +alan() int
        +cevre() int
    }
    Sekil <|-- Dikdortgen
    Sekil <|-- DikUcgen
    Dikdortgen <|-- Kare
    Boyanabilir <|.. Dikdortgen
```

Görevler

1. `Boyanabilir` interface'ini yazın.
2. Abstract `Sekil` sınıfını yazın. `bilgi()` şunu döndürsün: `ad + " | alan: " + alan() + " | çevre: " + cevre()`
3. `Dikdortgen` sınıfı: yapıcı `super("Dikdörtgen")` çağırsın. `renk` başlangıçta `"beyaz"` olsun. Alan `en * boy`, çevre `2 * (en + boy)`.
4. `Kare` sınıfı: yapıcı `super(kenar, kenar)` çağırsın, ardından `ad` alanını `"Kare"` yapsın. Başka metot yazmayın.
5. `DikUcgen` sınıfı: yapıcı `super("Dik üçgen")` çağırsın. Alan `a * b / 2`, çevre `a + b + c`.

Main'de yapılacaklar

1. `Sekil` tipinde üç değişken oluşturun: `Dikdortgen(4, 6)`, `Kare(5)`, `DikUcgen(3, 4, 5)`. Üçünün de `bilgi()` sonucunu yazdırın.
2. `Boyanabilir b = new Kare(3);` yazın. `"Önce: "` ile rengi yazdırın, `"mavi"` boyayın, `"Sonra: "` ile tekrar yazdırın.

Beklenen çıktı

```
Dikdörtgen | alan: 24 | çevre: 20
Kare | alan: 25 | çevre: 20
Dik üçgen | alan: 6 | çevre: 12
Önce: beyaz
Sonra: mavi
```

Düşün: `Kare` sınıfına hiç `alan()` yazmadınız, yine de çalıştı. Kare'nin alanını hangi sınıfın metodu hesaplıyor?

Senaryo 3: Banka

Bir banka üç tür hesap sunuyor. Para yatırmak her hesapta aynı, ama para çekme kuralı her hesapta farklı. Vadesiz hesap bakiyesi kadar, kredili hesap bakiye artı limiti kadar çekebilir. Vadeli hesaptan para çekilemez ama faiz eklenir.

```
classDiagram
    class Hesap {
        <<abstract>>
        #String sahip
        #int bakiye
        +Hesap(String sahip, int bakiye)
        +getSahip() String
        +paraYatir(int miktar) void
        +paraCek(int miktar)* boolean
        +bilgi() String
    }
    class Faizli {
        <<interface>>
        +faizEkle() void
    }
    class VadesizHesap {
        +VadesizHesap(String sahip, int bakiye)
        +paraCek(int miktar) boolean
    }
    class KrediliHesap {
        -int limit
```

```
        +KrediliHesap(String sahip, int bakiye, int limit)
        +paraCek(int miktar) boolean
    }
    class VadeliHesap {
        -int faizYuzde
        +VadeliHesap(String sahip, int bakiye, int faizYuzde)
        +paraCek(int miktar) boolean
        +faizEkle() void
    }
    Hesap <|-- VadesizHesap
    Hesap <|-- KrediliHesap
    Hesap <|-- VadeliHesap
    Faizli <|.. VadeliHesap
```

Görevler

1. `Faizli` interface'ini yazın.
2. Abstract `Hesap` sınıfını yazın. `paraYatir` bakiyeyi artırsın. `bilgi()` şunu döndürsün: `sahip + " - bakiye: " + bakiye + " TL"`
3. `VadesizHesap`: miktar bakiyeden küçük ya da eşitse bakiyeden düşüp `true`, değilse `false` döndürsün.
4. `KrediliHesap`: miktar `bakiye + limit` değerinden küçük ya da eşitse bakiyeden düşüp `true` döndürsün. Bakiye eksiye düşebilir.
5. `VadeliHesap`: `paraCek` her zaman `false` döndürsün. `faizEkle()` bakiyeye `bakiye * faizYuzde / 100` eklesin.
6. `Main` içine `static void cek(Hesap h, int miktar)` yazın. Çekim başarılıysa `"Ayşe 300 TL çekti."`, değilse `"Ayşe 900 TL çekemedi."` biçiminde yazsın. Ardından `h.bilgi()` yazdırsın.

Main'de yapılacaklar

1. Nesneler: `VadesizHesap("Ayşe", 1000)`, `KrediliHesap("Mehmet", 200, 500)`, `VadeliHesap("Zeynep", 5000, 10)`. İlk ikisini `Hesap` tipinde değişkene atın.
2. Sırayla çağırın: `cek(Ayşe, 300)`, `cek(Ayşe, 900)`, `cek(Mehmet, 600)`, `cek(Zeynep, 100)`.
3. Zeynep'in hesabını `Faizli` tipinde bir değişkene atayıp `faizEkle()` çağırın, sonra `bilgi()` yazdırın.

Beklenen çıktı

```
Ayşe 300 TL çekti.
Ayşe - bakiye: 700 TL
Ayşe 900 TL çekemedi.
Ayşe - bakiye: 700 TL
Mehmet 600 TL çekti.
Mehmet - bakiye: -400 TL
Zeynep 100 TL çekemedi.
Zeynep - bakiye: 5000 TL
Zeynep - bakiye: 5500 TL
```

Düşün: `cek` metodu parametre olarak `Hesap` alıyor ama üç farklı kural çalışıyor. Java hangi `paraCek` metodunu çalıştıracağına neye bakarak karar veriyor?

Senaryo 4: Ulaşım

Bir belediye araç filosunu takip ediyor. Her aracın plakası ve hızı var, teker sayısı ise araca göre değişiyor. Otomobil ve otobüs yolcu taşıyor, scooter ise elektrikli ve şarj ediliyor.

```
classDiagram
    class Arac {
        <<abstract>>
        #String plaka
        #int hiz
        +Arac(String plaka, int hiz)
        +tekerSayisi()* int
        +sure(int km) int
        +bilgi() String
    }
    class Elektrikli {
        <<interface>>
        +sarjEt(int yuzde) void
        +getSarj() int
    }
    class YolcuTasir {
        <<interface>>
        +kapasite() int
    }
    class Otomobil {
        +Otomobil(String plaka, int hiz)
        +tekerSayisi() int
        +kapasite() int
    }
    class Otobus {
        +Otobus(String plaka, int hiz)
        +tekerSayisi() int
```

```
        +kapasite() int
    }
    class ElektrikliScooter {
        -int sarj
        +ElektrikliScooter(String plaka, int hiz)
        +tekerSayisi() int
        +sarjEt(int yuzde) void
        +getSarj() int
    }
    Arac <|-- Otomobil
    Arac <|-- Otobus
    Arac <|-- ElektrikliScooter
    YolcuTasir <|.. Otomobil
    YolcuTasir <|.. Otobus
    Elektrikli <|.. ElektrikliScooter
```

Görevler

1. `Elektrikli` ve `YolcuTasir` interface'lerini yazın.
2. Abstract `Arac` sınıfını yazın. `sure(km)` dakika cinsinden `km * 60 / hiz` döndürsün. `bilgi()` şunu döndürsün: `plaka + " | " + tekerSayisi() + " teker | " + hiz + " km/sa"`
3. `Otomobil` : 4 teker, 5 yolcu. `Otobus` : 6 teker, 50 yolcu.
4. `ElektrikliScooter` : 2 teker. `sarj` başlangıçta 20 olsun. `sarjEt` şarjı artırsın ama 100'ü geçirmesin.
5. `Main` içine `static void yaz(Arac a)` yazın. Şunu yazdırsın: `a.bilgi() + " | 30 km: " + a.sure(30) + " dk"`

Main'de yapılacaklar

1. Nesneler: `Otomobil("34 ABC 34", 90)` , `Otobus("06 OTO 06", 60)` , `ElektrikliScooter("SCT-7", 20)` . Üçünü de `yaz` metoduna gönderin.
2. Otomobil ve otobüsü `YolcuTasir` tipinde iki değişkene atın. Kapasitelerin toplamını `"Toplam yolcu kapasitesi: "` ile yazdırın.
3. Scooter'ı `Elektrikli` tipinde bir değişkene atın. İki kez `sarjEt(50)` çağırın, her seferinden sonra `"Şarj: %"` ile şarjı yazdırın.

Beklenen çıktı

```
34 ABC 34 | 4 teker | 90 km/sa | 30 km: 20 dk
06 OTO 06 | 6 teker | 60 km/sa | 30 km: 30 dk
SCT-7 | 2 teker | 20 km/sa | 30 km: 90 dk
Toplam yolcu kapasitesi: 55
Şarj: %70
Şarj: %100
```

Düşün: `Elektrikli e` değişkeni üzerinden `e.bilgi()` çağırmayı deneyin. Nesne aynı scooter olduğu halde neden derlenmiyor?

Senaryo 5: Kahramanlar Loncası

Bir oyunda üç kahraman birbirine saldırıyor. Her kahramanın özel saldırısı farklı güçte. Savaşçının kalkanı gelen hasarın bir kısmını emiyor, şifacı ise arkadaşlarını iyileştirebiliyor.

```
classDiagram
    class Karakter {
        <<abstract>>
        #String ad
        #int can
        #int guc
        +Karakter(String ad, int can, int guc)
        +getAd() String
        +ozelSaldiri()* int
        +hasarAl(int miktar) void
        +durum() String
    }
    class Kalkanli {
        <<interface>>
        +kalkanGucu() int
    }
    class Iyilestirici {
        <<interface>>
        +iyilestir(Karakter hedef) void
    }
    class Savasci {
        +Savasci(String ad, int can, int guc)
        +ozelSaldiri() int
        +kalkanGucu() int
        +hasarAl(int miktar) void
    }
    class Buyucu {
        +Buyucu(String ad, int can, int guc)
        +ozelSaldiri() int
    }
```

```
    class Sifaci {
        +Sifaci(String ad, int can, int guc)
        +ozelSaldiri() int
        +iyilestir(Karakter hedef) void
    }
    Karakter <|-- Savasci
    Karakter <|-- Buyucu
    Karakter <|-- Sifaci
    Kalkanli <|.. Savasci
    Iyilestirici <|.. Sifaci
```

Görevler

1. `Kalkanli` ve `Iyilestirici` interface'lerini yazın.
2. Abstract `Karakter` sınıfını yazın. `hasarAl` canı miktar kadar azaltsın, can 0'ın altına düşerse 0 yapsın. `durum()` şunu döndürsün: `ad + " [can: " + can + "]"`
3. `Savasci`: özel saldırı `guc * 2`, kalkan gücü 5. `hasarAl` metodunu ezin: gelen miktardan kalkan gücünü çıkarın (sonuç eksiyse 0 yapın) ve kalanı `super.hasarAl(...)` ile gönderin.
4. `Buyucu`: özel saldırı `guc * 3`.
5. `Sifaci`: özel saldırı `guc`. `iyilestir(hedef)` hedefin canını 20 artırsın.
6. `Main` içine `static void saldir(Karakter saldiran, Karakter hedef)` yazın. Saldıranın özel saldırısını hedefe uygulasın, sonra `"Yıldız -> Demir : 36 hasar"` biçiminde bir satır ve hedefin `durum()` sonucunu yazdırsın.

Main'de yapılacaklar

1. Nesneler: `Savasci("Demir", 100, 15)`, `Buyucu("Yıldız", 70, 12)`, `Sifaci("Şifa", 80, 5)`
2. Sırayla: büyücü savaşçıya, savaşçı büyücüye, şifacı büyücüye saldırsın.
3. Şifacıyı `Iyilestirici` tipinde bir değişkene atın. Önce savaşçıyı, sonra büyücüyü iyileştirin ve ikisinin `durum()` sonucunu yazdırın.

Beklenen çıktı

```
Yıldız -> Demir : 36 hasar
Demir [can: 69]
Demir -> Yıldız : 30 hasar
Yıldız [can: 40]
Şifa -> Yıldız : 5 hasar
Yıldız [can: 35]
Demir [can: 89]
Yıldız [can: 55]
```

Düşün: Büyücü 36 hasar vurdu ama Demir'in canı yalnızca 31 azaldı. `saldir` metodunda kalkanla ilgili hiçbir şey yazmadınız; kalkan nasıl devreye girdi?

Senaryo 6: Okul Personeli

Okul, personelin maaş bordrosunu hazırlayan bir program istiyor. Her personelin maaşı farklı hesaplanıyor. Öğretmenler nöbet tutuyor, yöneticiler prim alıyor. Müdür yardımcısı hem öğretmen hem yönetici olduğu için ikisini de yapıyor.

```
classDiagram
    class Personel {
        <<abstract>>
        #String ad
        +Personel(String ad)
        +maasHesapla()* int
        +bordro() String
    }
    class Nobetci {
        <<interface>>
        +nobetTut(String gun) String
    }
    class Primli {
        <<interface>>
        +prim() int
    }
    class Ogretmen {
        #int dersSaati
        +Ogretmen(String ad, int dersSaati)
        +maasHesapla() int
        +nobetTut(String gun) String
    }
    class MudurYardimcisi {
        +MudurYardimcisi(String ad, int dersSaati)
        +prim() int
        +maasHesapla() int
    }
    class Mudur {
        +Mudur(String ad)
        +prim() int
        +maasHesapla() int
    }
    class Memur {
        +Memur(String ad)
        +maasHesapla() int
    }
```

```
    Personel <|-- Ogretmen
    Personel <|-- Mudur
    Personel <|-- Memur
    Ogretmen <|-- MudurYardimcisi
    Nobetci <|.. Ogretmen
    Primli <|.. Mudur
    Primli <|.. MudurYardimcisi
```

Görevler

1. `Nobetci` ve `Primli` interface'lerini yazın.
2. Abstract `Personel` sınıfını yazın. `bordro()` şunu döndürsün: `ad + " : " + maasHesapla() + " TL"`
3. `Ogretmen`: maaş `30000 + dersSaati * 500`. `nobetTut(gun)` şunu döndürsün: `ad + " " + gun + " günü nöbetçi."`
4. `MudurYardimcisi`: prim 4000. Maaşı `super.maasHesapla() + prim()` olsun. Nöbet metodunu yeniden yazmayın.
5. `Mudur`: prim 8000, maaş `50000 + prim()`.
6. `Memur`: maaş 28000.
7. `Main` içine `static int toplam = 0;` ve `static void ode(Personel p)` yazın. `ode`, bordroyu yazdırıp maaşı `toplam`'a eklesin.

Main'de yapılacaklar

1. Nesneler: `Ogretmen("Ali Hoca", 20)`, `MudurYardimcisi("Elif Hanım", 10)`, `Mudur("Selin Hanım")`, `Memur("Hasan Bey")`. Dördünü bu sırayla `ode` metoduna gönderin.
2. `"Toplam maaş: "` ile toplamı yazdırın.
3. Öğretmeni ve müdür yardımcısını `Nobetci` tipinde değişkenlere atın. Birini `"Pazartesi"`, diğerini `"Cuma"` günü nöbete yazın.
4. Müdürü ve müdür yardımcısını `Primli` tipinde değişkenlere atın. Primlerin toplamını `"Toplam prim: "` ile yazdırın.

Beklenen çıktı

```
Ali Hoca : 40000 TL
Elif Hanım : 39000 TL
Selin Hanım : 58000 TL
Hasan Bey : 28000 TL
Toplam maaş: 165000 TL
Ali Hoca Pazartesi günü nöbetçi.
Elif Hanım Cuma günü nöbetçi.
Toplam prim: 12000 TL
```

Düşün: `MudurYardimcisi` sınıfına `implements Nobetci` yazmadınız, ama `Nobetci` tipindeki değişkene atanabildi. Neden?

Senaryo 7: Akıllı Ev

Bir akıllı ev uygulaması cihazların açık olup olmadığını ve ne kadar enerji harcadığını gösteriyor. Lambanın parlaklığı, klimanın sıcaklığı ayarlanabiliyor. Klima sesli komutla da çalışıyor. Buzdolabı hep açık.

```
classDiagram
    class Cihaz {
        <<abstract>>
        #String ad
        #boolean acik
        +Cihaz(String ad)
        +ac() void
        +kapat() void
        +saatlikTuketim()* int
        +tuketim(int saat) int
        +durum() String
    }
    class Ayarlanabilir {
        <<interface>>
        +ayarla(int deger) void
    }
    class SesliKomut {
        <<interface>>
        +komutAl(String komut) String
    }
    class Lamba {
        -int parlaklik
        +Lamba(String ad)
        +saatlikTuketim() int
        +ayarla(int deger) void
    }
    class Klima {
        -int sicaklik
        +Klima(String ad)
        +saatlikTuketim() int
        +ayarla(int deger) void
        +komutAl(String komut) String
    }
    class Buzdolabi {
        +Buzdolabi(String ad)
        +saatlikTuketim() int
```

```
    }
    Cihaz <|-- Lamba
    Cihaz <|-- Klima
    Cihaz <|-- Buzdolabi
    Ayarlanabilir <|.. Lamba
    Ayarlanabilir <|.. Klima
    SesliKomut <|.. Klima
```

Görevler

1. `Ayarlanabilir` ve `SesliKomut` interface'lerini yazın.
2. Abstract `Cihaz` sınıfını yazın. `acik` başlangıçta `false`. `tuketim(saat)` cihaz kapalıysa 0, açıksa `saatlikTuketim() * saat` döndürsün. `durum()` açıksa `ad + " AÇIK"`, kapalıysa `ad + " KAPALI"` döndürsün.
3. `Lamba`: `parlaklik` başlangıçta 50. Saatlik tüketim `parlaklik / 5`. `ayarla` parlaklığı değiştirsin.
4. `Klima`: `sicaklik` başlangıçta 24, saatlik tüketim 1000. `ayarla` sıcaklığı değiştirsin. `komutAl`:
   - `"aç"` gelirse `ac()` çağırıp `ad + " açıldı, " + sicaklik + " derece."` döndürsün.
   - `"kapat"` gelirse `kapat()` çağırıp `ad + " kapandı."` döndürsün.
   - Başka bir şey gelirse `"Komut anlaşılmadı."` döndürsün. Karşılaştırma için `komut.equals("aç")` kullanın.
5. `Buzdolabi`: saatlik tüketim 150. Yapıcısı `super(ad)` sonrasında `ac()` çağırsın.
6. `Main` içine `static void rapor(Cihaz c)` yazın: `c.durum() + " | 5 saatte " + c.tuketim(5) + " Wh"`

Main'de yapılacaklar

1. Nesneler: `Lamba("Salon lambası")`, `Klima("Yatak odası kliması")`, `Buzdolabi("Buzdolabı")`. Üçü için `rapor` çağırın.
2. Lambayı açın. Lambayı `Ayarlanabilir` tipinde bir değişkene atıp 80'e ayarlayın. Klimayı da `Ayarlanabilir` tipinde bir değişkene atıp 21'e ayarlayın.
3. Klimayı `SesliKomut` tipinde bir değişkene atın. `"aç"` ve `"dans et"` komutlarını gönderip sonuçları yazdırın.
4. Üç cihaz için yeniden `rapor` çağırın.

Beklenen çıktı

```
Salon lambası KAPALI | 5 saatte 0 Wh
Yatak odası kliması KAPALI | 5 saatte 0 Wh
Buzdolabı AÇIK | 5 saatte 750 Wh
Yatak odası kliması açıldı, 21 derece.
Komut anlaşılmadı.
Salon lambası AÇIK | 5 saatte 80 Wh
Yatak odası kliması AÇIK | 5 saatte 5000 Wh
Buzdolabı AÇIK | 5 saatte 750 Wh
```

Düşün: Bir `Kamera` cihazı eklemek isteseydiniz `Cihaz` sınıfını ya da `rapor` metodunu değiştirmeniz gerekir miydi? Kamera için en az neyi yazmanız gerekirdi?

Senaryo 8: Restoran

Bir restoran sipariş fişi yazdıran bir program istiyor. Her yemeğin adı ve fiyatı var, hazırlanma süresi yemeğe göre değişiyor. Bazı yemeklerde indirim var, bazıları paket servise uygun.

```
classDiagram
    class Yemek {
        <<abstract>>
        #String ad
        #int fiyat
        +Yemek(String ad, int fiyat)
        +getFiyat() int
        +hazirlanmaSuresi()* int
        +fis() String
    }
    class Indirimli {
        <<interface>>
        +indirimUygula(int yuzde) void
    }
    class Paketlenebilir {
        <<interface>>
        +paketUcreti() int
    }
    class Pizza {
        +Pizza(String ad, int fiyat)
        +hazirlanmaSuresi() int
        +paketUcreti() int
    }
    class Burger {
        +Burger(String ad, int fiyat)
        +hazirlanmaSuresi() int
        +indirimUygula(int yuzde) void
```

```
        +paketUcreti() int
    }
    class Salata {
        +Salata(String ad, int fiyat)
        +hazirlanmaSuresi() int
        +indirimUygula(int yuzde) void
    }
    Yemek <|-- Pizza
    Yemek <|-- Burger
    Yemek <|-- Salata
    Paketlenebilir <|.. Pizza
    Paketlenebilir <|.. Burger
    Indirimli <|.. Burger
    Indirimli <|.. Salata
```

Görevler

1. `Indirimli` ve `Paketlenebilir` interface'lerini yazın.
2. Abstract `Yemek` sınıfını yazın. `fis()` şunu döndürsün: `ad + " - " + fiyat + " TL - " + hazirlanmaSuresi() + " dk"`
3. `Pizza`: 15 dk, paket ücreti 20.
4. `Burger`: 10 dk, paket ücreti 15. `indirimUygula(yuzde)` fiyattan `fiyat * yuzde / 100` düşsün.
5. `Salata`: 5 dk, indirim Burger'deki gibi.
6. `Main` içine `static int toplam = 0;` ve `static void siparis(Yemek y)` yazın. Fişi yazdırıp fiyatı `toplam`'a eklesin.

Main'de yapılacaklar

1. Nesneler: `Pizza("Karışık pizza", 250)`, `Burger("Köfte burger", 200)`, `Salata("Çoban salata", 80)`
2. Burger'i ve salatayı `Indirimli` tipinde değişkenlere atın. Burger'e yüzde 25, salataya yüzde 10 indirim uygulayın.
3. Üç yemeği de `siparis` metoduna gönderin. `"Restoranda toplam: "` ile toplamı yazdırın.
4. Pizzayı ve burger'i `Paketlenebilir` tipinde değişkenlere atın. Toplama iki paket ücretini ekleyip `"Paket servis toplamı: "` ile yazdırın.

Beklenen çıktı

```
Karışık pizza - 250 TL - 15 dk
Köfte burger - 150 TL - 10 dk
Çoban salata - 72 TL - 5 dk
Restoranda toplam: 472 TL
Paket servis toplamı: 507 TL
```

Düşün: `Indirimli i = new Pizza("Sucuklu", 300);` satırı derlenir mi? Neden?

Senaryo 9: Orkestra

Okul orkestrası konser öncesi çalgıları akort edip sırayla çalıyor. Her çalgının sesi farklı. Elektro gitar, gitarın her özelliğini taşıyor ama ayrıca amfiye bağlanıyor. Davul akort edilmez, derisi gerilir.

```
classDiagram
    class Calgi {
        <<abstract>>
        #String ad
        +Calgi(String ad)
        +getAd() String
        +cal()* String
        +akortEt() String
    }
    class Telli {
        <<interface>>
        +telSayisi() int
    }
    class Elektronik {
        <<interface>>
        +fisTak() String
    }
    class Gitar {
        +Gitar(String ad)
        +cal() String
        +telSayisi() int
    }
    class ElektroGitar {
        +ElektroGitar(String ad)
        +cal() String
        +fisTak() String
    }
    class Keman {
        +Keman(String ad)
        +cal() String
        +telSayisi() int
```

```
    }
    class Davul {
        +Davul(String ad)
        +cal() String
        +akortEt() String
    }
    Calgi <|-- Gitar
    Calgi <|-- Keman
    Calgi <|-- Davul
    Gitar <|-- ElektroGitar
    Telli <|.. Gitar
    Telli <|.. Keman
    Elektronik <|.. ElektroGitar
```

Görevler

1. `Telli` ve `Elektronik` interface'lerini yazın.
2. Abstract `Calgi` sınıfını yazın. `akortEt()` şunu döndürsün: `ad + " akort edildi."`
3. `Gitar`: `cal()` → `"Tın tın tın"`, 6 tel.
4. `ElektroGitar`: `cal()` metodunu ezin ve `super.cal() + " (amfiden, çok yüksek!)"` döndürün. `fisTak()` → `ad + " amfiye bağlandı."`
5. `Keman`: `cal()` → `"Iiiiyy"`, 4 tel.
6. `Davul`: `cal()` → `"Bum bum tıss"`. `akortEt()` metodunu ezin: `ad + ": deri gerildi."`
7. `Main` içine `static void konser(Calgi c)` yazın. Önce `c.akortEt()`, sonra `c.getAd() + ": " + c.cal()` yazdırsın.

Main'de yapılacaklar

1. Nesneler: `Gitar("Klasik gitar")`, `ElektroGitar("Elektro gitar")`, `Keman("Keman")`, `Davul("Davul")`
2. Önce elektro gitarın fişini takıp sonucu yazdırın. Sonra dört çalgıyı bu sırayla `konser` metoduna gönderin.
3. Klasik gitarı, elektro gitarı ve kemanı `Telli` tipinde değişkenlere atın. Tel sayılarının toplamını `"Toplam tel: "` ile yazdırın.

Beklenen çıktı

```
Elektro gitar amfiye bağlandı.
Klasik gitar akort edildi.
Klasik gitar: Tın tın tın
Elektro gitar akort edildi.
Elektro gitar: Tın tın tın (amfiden, çok yüksek!)
Keman akort edildi.
Keman: Iiiiyy
Davul: deri gerildi.
Davul: Bum bum tıss
Toplam tel: 16
```

Düşün: `ElektroGitar` sınıfına `telSayisi()` yazmadınız. Elektro gitarın 6 teli nereden geldi?

Senaryo 10: Kütüphane

Okul kütüphanesi kitap, e-kitap ve dergi tutuyor. Her materyalin ödünç süresi farklı. Bir kitabı aynı anda yalnızca bir kişi alabilir, e-kitabı ise herkes alabilir ve indirebilir. Dergiler ödünç verilmez.

```
classDiagram
    class Materyal {
        <<abstract>>
        #String baslik
        #int yil
        +Materyal(String baslik, int yil)
        +oduncGunu()* int
        +bilgi() String
    }
    class OduncVerilebilir {
        <<interface>>
        +oduncVer(String kisi) boolean
        +iadeEt() void
    }
    class Dijital {
        <<interface>>
        +indir() String
    }
    class Kitap {
        -String alan
        +Kitap(String baslik, int yil)
        +oduncGunu() int
        +oduncVer(String kisi) boolean
        +iadeEt() void
    }
```

```
class EKitap {
    +EKitap(String baslik, int yil)
    +oduncGunu() int
    +oduncVer(String kisi) boolean
    +indir() String
}
class Dergi {
    +Dergi(String baslik, int yil)
    +oduncGunu() int
}
Materyal <|-- Kitap
Materyal <|-- Dergi
Kitap <|-- EKitap
OduncVerilebilir <|.. Kitap
Dijital <|.. EKitap
```

Görevler

1. `OduncVerilebilir` ve `Dijital` interface'lerini yazın.
2. Abstract `Materyal` sınıfını yazın. `bilgi()` şunu döndürsün: `baslik + " (" + yil + ")"`
3. `Kitap`: 15 gün. `alan` başlangıçta `null`. `oduncVer(kisi)`: `alan` boşsa kişiyi yazıp `true`, doluysa `false` döndürsün. `iadeEt()` `alan`'ı yeniden `null` yapsın.
4. `EKitap`: 30 gün. `oduncVer` metodunu ezin, her zaman `true` döndürsün. `indir()` → `baslik + " indiriliyor..."`
5. `Dergi`: 0 gün.
6. `Main` içine iki metot yazın:
   - `static void goster(Materyal m)`: `m.bilgi() + " -> ödünç süresi: " + m.oduncGunu() + " gün"`
   - `static void ver(OduncVerilebilir o, String kisi)`: başarılıysa `kisi + " ödünç aldı."`, değilse `kisi + " alamadı, materyal başkasında."`

Main'de yapılacaklar

1. Nesneler: `Kitap("Suç ve Ceza", 1866)`, `EKitap("Java'ya Giriş", 2025)`, `Dergi("Bilim Dünyası", 2026)`. Üçü için `goster` çağırın.
2. Kitabı önce Ece'ye, sonra Can'a vermeyi deneyin. Kitap iade edilsin, Can'a yeniden verin.
3. E-kitabı önce Ece'ye, sonra Can'a verin.
4. E-kitabı `Dijital` tipinde bir değişkene atıp `indir()` sonucunu yazdırın.

Beklenen çıktı

```
Suç ve Ceza (1866) -> ödünç süresi: 15 gün
Java'ya Giriş (2025) -> ödünç süresi: 30 gün
Bilim Dünyası (2026) -> ödünç süresi: 0 gün
Ece ödünç aldı.
Can alamadı, materyal başkasında.
Can ödünç aldı.
Ece ödünç aldı.
Can ödünç aldı.
Java'ya Giriş indiriliyor...
```

Düşün: `ver(dergi, "Ece")` yazmayı deneyin. Derleyici neden kabul etmiyor? Dergiyi ödünç verilebilir yapmak için ne değiştirmeniz gerekir?